Elevator Simulation Design Document, vFinal

Team Name: Updog
Team Members: Arnav and Sarp

1. **GUI Mockup**

   This can be hand-drawn or designed in slides or drawIO - but paste a picture here. It should be clear how you will convey all of the required information and controls to the users.

2. **Identify the ownership of each class:**

   Arnav
   Default Package: ElevatorSimController.java, ElevatorSimulation.java
   Building package: CallManager.java, State.java
   passengers package: Passengers.javalic

   Sarp
   Building package:

Building Package: Building.java, Elevator.java, Floor.java

3. **Identify (first pass) all of the externally visible methods (public and package level) *other than Getters/Setters* that you think you will need for each class.**

ElevatorSimController:
// Constructor instantiates a new instance of the Elevator Sim Controller
public ElevatorSimController(ElevatorSimulation gui)
// Method enables logging for the Elevator Sim Controller instance
public void enableLogging()
// Goes through each step of the simulation, incrementing the time during each pass
public void stepSim() {
// Prints out everyone currently in the queue
public void dumpPassQ()

ElevatorSimulation:
// Instantiates a new elevator simulation
public ElevatorSimulation()
// Starts the simulation
public void start(Stage primaryStage) throws Exception
// Allows the the user to change millisPerTick through the command line
public static void main (String[] args)

Building:
// Instantiates a new building.
public Building(int numFloors, String logfile)
// Creates and attaches a new Elevator instance to this building.
public void createElevator(int capacity, int floorTicks, int doorTicks, int passPerTick)
// Notifies the CallManager that all passengers for this tick have been added.
public void allPassengersAddedForTick()
// Adds a new passenger group to the appropriate floor queue and logs the call.
public void addPassengerToFloor(Passengers p, int time)
// Updates the elevator's state and position for the current tick.
public void updateElevator(int time)
// Writes all successful arrivals and give-ups to a CSV file.
public void processPassengerData()
// Enables logging and prints the initial elevator configuration.
public void enableLogging()
// Closes the log file and logs the end of the simulation.
public void closeLogs(int time)
// Checks if there is a call on the specified floor in the given direction.
public boolean isCallAtFloorDirection(int floor, int dir)
// Returns the total number of passengers waiting on the given floor.
public int getNumPassengers(int floor)

// Returns true if the elevator is currently in the STOP state.
public boolean elevatorIsStopped()
// Reset static ID counter for consistent Passenger IDs across test runs.
public static void resetStaticID()
// Returns a formatted string representation of this passenger group.
public String toString()

CallManager:
// Instantiates a new call manager.
public CallManager(Floor[] floors, int numFloors)
// Update call status.
void updateCallStatus()
// Prioritize passenger calls from STOP STATE
Passengers prioritizePassengerCalls(int floor)
// Sets the polite flag for the next passenger in the queue for the given elevator and floor.
public int makePolite()
// Determines if there are passengers to board at the given floor
public boolean passengersToBoard(Elevator elevator, Floor floor)
// Boards passengers onto the elevator from the given floor.
public Passengers boardPassengers(Elevator elevator, Floor floor, Passengers passengers)
// Returns true if there are any pending calls (up or down) in the building.
// Peeks at the next passenger in the queue for the given direction and floor
public Passengers peekQueue(int direction, int floor)
// Removes and returns the next passenger in the queue for the given direction and floor
public Passengers removeQueue(int direction, int floor)
// Returns true if there are any pending calls (up or down) in the building.
boolean callPending()
// Check whether there are up calls above the given floor.
boolean hasUpCallsAbove(int floor)
// Check whether there are up calls below the given floor.
boolean hasUpCallsBelow(int floor)
// Check whether there are down calls below the given floor.
boolean hasDownCallsBelow(int floor)
// Check whether there are down calls above the given floor.
boolean hasDownCallsAbove(int floor)
// Is there a call on the specified floor in the given direction
boolean isCallAtFloorDirection(int floor, int dir)

Elevator:
// Instantiates a new elevator.
public Elevator(int numFloors, int capacity, int floorTicks, int doorTicks, int passPerTick)
// Update floor position based on current state and time in state
public void updateFloor()
// Update door state (opening or closing) based on current state
public void updateDoor()
// Transition to the next state and update time-in-state counter
void updateCurrState(ElStates nextState)

// Determines if there are passengers scheduled to get off at the current floor
public boolean passengersToGetOff()
// Set whether the elevator skipped a floor
public void skip(boolean skip)
// Check if the elevator is currently at a floor
public boolean isAtFloor()
// Increase passenger delay counter based on boarding or exiting passengers
public void increaseDelay(boolean ifBoarding)
// Check if the current passenger delay (boarding/exiting) is complete
public boolean delayDone()
// Attempt to board passengers from the current floor queue
public Passengers board(Floor floor, CallManager callMgr)
// Add a passenger group to the elevator's destination list and update board time
public void addPassenger(Passengers p, int currentTime)
// Remove and return the next passenger group scheduled to exit at the given floor
public Passengers removePassengerAtFloor(int floor, int currentTime)

Floor:
// Instantiates a new Floor, with a specified up and down queue length
public Floor(int qSize)
// Returns a string representation of the contents of the specified queue
String queueString(int dir)
// Adds a passenger group to the up queue for this floor.
public void addToUpQueue(Passengers p)
// Adds a passenger group to the down queue for this floor.
public void addToDownQueue(Passengers p)
// Returns the next passenger group in the up queue without removing it
public Passengers peekUpQueue()
// Returns the next passenger group in the down queue without removing it
public Passengers peekDownQueue()
// Removes and returns the next passenger group from the up queue
public Passengers removeUpQueue()
// Removes and returns the next passenger group from the down queue
public Passengers removeDownQueue()
// Removes and returns the next passenger group from the up queue
public Passengers pollUpQueue()
// Removes and returns the next passenger group from the down queue
public Passengers pollDownQueue()

Passenger:
// Instantiates a new passenger
public Passengers(int time, int numPass, int on, int dest, boolean polite, int waitTime)
// Gets a formatted string for the passenger instance
public String toString()

State:
// return the currState

public ElStates getCurrState()
// return the currFloor
public int getCurrFloor()
// return the numPassengers
public int getNumPassengers()
// Constructor for State
State(ElStates currState, int currFloor, int numPassengers)

4. **Project Management, Work Flow, Conflict Resolution and Milestones**

For the first pass of this document, you should answer the following question:
- how will you manage/communicate turning in new code to your shared repository,
*- what actions will you take should there be a problem **before** coming to talk with me?*

You should also have a placeholder for a schedule of milestones - this will be completed separately, but should be pasted in when done. Using Google Sheets, or a table, is a good way of doing this.

We will ensure communication between Sarp and I, in and out of school, and express ideas openly. ***I will email, rather than text Sarp for communications regarding the elevator project, to ensure he is able to see my messages. I can also use Google Chat if I have an urgent message, as I know he uses it more often. However, I will avoid calling/texting him as limitations on screen time hinder communication that way.*** *If we have issues or conflicts of interest, we will communicate our thoughts to each other respectfully and try to combine our different ideas to contribute effectively to the project. We will always go over our goals and what we plan to accomplish every class and whenever we work on the project together so that we can both have a grasp on what the other is doing and how the project is going. If a code-related problem arises, we will attempt to debug it together; however, we will realize when we are stuck and need help. If a disagreement arises, we will also attempt to resolve this between ourselves, but will talk to Mr. Murray if we need help resolving it.*

Green = finished

Red = unfinished tasks to complete

Milestone	Owner	Date
Passenger class completed	Arnav	Dec 4
Basic static GUI Layout	Arnav	Dec 5
Floor class completed	Arnav	Dec 5
Finishing building class	Sarp	Dec 8

basic Stepsim, incrementing time, etc.	Arnav	Dec 8
First finished draft	Both	Dec 11
FSM correctly transitions	Sarp	Dec 11
Correct state transitions and elevator logic, passing Basic, Mv1Flr, MvToFlr	Both	Dec 16
Passing all tests 100%	Sarp	Jan 6
Completed GUI implementation	Both	Jan 6
AnalyzeCode Passing	Arnav	Jan 13